

# altairsim — Have an AI find and fix a bug — over MCP

---

2026-08-23 · f274e98

This folder is a working directory for an **AI assistant driving altairsim over MCP**. It holds a CP/M Altair and a tiny program, `HELLO.ASM`, that is supposed to print `HELLO, WORLD` and does not. There is one deliberate bug in it. The exercise is that the assistant — not you — assembles it, runs it, sees the wrong output, single-steps the loop to find the fault, corrects the source, and reassembles, **all through the simulator's MCP tools**. No cutting and pasting.

If you have never wired an assistant to altairsim, read `DRIVING-WITH-AI.md` (it ships beside this folder) first — it is the briefing you drop in front of the assistant, and it explains the MCP server and the one registration step that this walkthrough assumes you have done.

## Point your assistant at this machine

The machine is `cpm-ai.toml`. Register the server **from this directory**, so the host bridge (`R/W`, below) can reach the source the assistant edits:

```
cd examples/ai-mcp
claude mcp add altairsim -- altairsim cpm-ai.toml --mcp      # Claude Code; see DRIVING-WITH-AI.md for other
clients
```

This is the `claude` command line, which runs the server here in this folder, so the relative `cpm-ai.toml` and the host bridge both resolve to it. (The Claude Desktop app starts the server in your home directory instead — there, use an absolute path; `DRIVING-WITH-AI.md` covers it, along with running several machines at once.)

Then start your assistant in this directory and give it the whole job in one sentence:

*Using the altairsim MCP tools, boot the CP/M machine and assemble and run `HELLO.ASM` off its disk. It should print `HELLO, WORLD` — if it doesn't, debug it in the simulator and fix the source.*

Say **"using the altairsim MCP tools"** and **"off its disk"** on purpose. `HELLO.ASM` lives on the CP/M disk, reached through the MCP `run / monitor` tools — *not* as a host file to open. If you just say "fix `HELLO.ASM`," a fresh assistant is apt to `find` it on your machine and start editing a host copy instead of driving the simulator — and when you are running from a checkout of this repository (rather than a

shipped package) there really is a pristine `HELLO.ASM` sitting loose in this folder for it to land on, which is not the file the exercise is about. Point it at the disk and that confusion never starts.

**Want to watch it work live?** Add `--mirror socket:2323` to the registered command (`... altairsim cpm-ai.toml --mcp --mirror socket:2323`), then `telnet localhost 2323` in another window while the assistant runs. You see the same console it is driving, character by character, and can type onto the line to take the keyboard yourself. `--mirror socket:2323?ro` watches without touching it. `DRIVING-WITH-AI.md` explains it under *Watching over its shoulder*.

The rest of this file is the session that unfolds, so you can follow along or check its work.

## 1 — Build it and watch it misbehave

The disk already carries `HELLO.ASM`, CP/M's `ASM` and `LOAD`, and the host-bridge utilities. The assistant boots the machine (`run {from: 0xFF00}` — under `--mcp` the startup is not run, so it boots the disk itself), then assembles, loads and runs the program:

```
run {input: "ASM HELLO\r", until: "A>"}      -> END OF ASSEMBLY
run {input: "LOAD HELLO\r", until: "A>"}      -> FIRST ADDRESS 0100 ... HELLO.COM
run {input: "HELLO\r", until: "A>"}          -> ELLO, WORLD
```

There it is: **ELLO, WORLD**. The `H` is missing. It assembled cleanly and ran without crashing, so this is a logic bug, not a typo the assembler could catch — exactly the kind of thing the debugger is for.

## 2 — Stop where the program loads, and look

The assistant sets a breakpoint at `0100` — the CP/M transient program area, where `HELLO.COM` loads — and runs it. CP/M's loader jumps to `0100` and the breakpoint trips before a single instruction of the program has run:

```
monitor {command: "BREAK 0x100"}              -> breakpoint 1: pc 0100
run      {input: "HELLO\r", until: "0100"}     -> stopped at breakpoint
monitor {command: "DISASM 0x100 6"}
```

```
0100 21 15 01 LXI H,0115      ; HL -> the string
0103 23      INX H           ; advance the pointer
0104 7E      MOV A,M         ; fetch the character
0105 B7      ORA A           ; the 0 terminator?
0106 CA 14 01 JZ 0114
0109 5F      MOV E,A
```

The string is at 0115, and 0115 holds 48 — 'H'. But look at the order: INX H at 0103 runs **before** MOV A,M at 0104. The pointer is advanced past the first character before that character is ever fetched. Single-stepping proves it — watch the A register:

```
monitor {command: "STEP 3"}
```

```
... H=0115 ... P=0103 INX H      ; HL points at 'H' (0115)
... H=0116 ... P=0104 MOV A,M    ; INX H has already moved it to 0116
... A=45   ... P=0105 ORA A      ; so A = 45 = 'E', never 'H'
```

A=45 is 'E'. The very first character the program prints is the second character of the string. The bug is an ordering slip: the loop increments the pointer at the top, before the load, when it should increment at the bottom, after the character has been sent.

### 3 — Fix the source, and reassemble

The fix is to move INX H from the top of the loop to the bottom, just before the JMP back — so the pointer advances *after* the character has been sent, not before it is fetched. The source lives on the disk, so the assistant first pulls it to a host file it can edit (W, the other half of the host bridge), corrects it, then copies it back (R) and reassembles:

```
run {input: "W HELLO.ASM HELLO.ASM T\r", until: "A>"} ; CP/M -> host (T = text, trims ^Z)
```

It edits the host HELLO.ASM — **keeping the file CR/LF**, or ASM will assemble it to nothing — so the loop reads:

```
LOOP:  MOV A,M      ;fetch the next character
        ORA A        ;the 0 terminator?
        JZ  DONE     ;yes -- finished
        MOV E,A      ;no -- character to E for BDOS
        MVI C,CONOUT
        PUSH H
        CALL BDOS
        POP H
        INX H        ;NOW advance -- after this character is sent
        JMP LOOP
```

then copies it back onto the disk and rebuilds:

```
run {input: "R HELLO.ASM\r",   until: "A>"}    ; host file -> CP/M (over the host bridge)
run {input: "ASM HELLO\r",     until: "A>"}    ; -> a new HELLO.HEX
run {input: "LOAD HELLO\r",    until: "A>"}    ; -> a new HELLO.COM
run {input: "HELLO\r",         until: "A>"}    -> HELLO, WORLD
```

HELLO, WORLD . The program was assembled, run, debugged, corrected and reassembled without the machine ever leaving the assistant's hands — which is the whole point.

## The host bridge, in one line

**R** and **W** are how a file crosses between your directory and the CP/M disk — they are what let the assistant edit `HELLO.ASM` here and build it in there:

- **R** `<hostfile> [cpmfile]` — host → CP/M. **W** `<cpmfile> [hostfile] [B|T]` — CP/M → host.
- **B (binary) is the default**, and is what a `.COM` needs. Use **T** for text ( `.PRN` , `.HEX` , `.ASM` ) so the trailing `^Z` padding is trimmed. Source must be **CR/LF** — `ASM` assembles an LF-only file to nothing.

## The files

| File                          | What it is                                                                                                                                                                                                                                                                                                                             |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cpm-ai.toml</code>      | The machine: <code>base = "default"</code> (the CP/M Altair) plus this folder's disk in drive 0.                                                                                                                                                                                                                                       |
| <code>cpm22b23-56k.dsk</code> | Mike Douglas's track-buffered CP/M 2.2b, the same image <code>examples/cpm</code> boots — with the buggy <code>HELLO.ASM</code> dropped on it. Also carries <code>ASM</code> , <code>LOAD</code> , <code>DDT</code> and the host-bridge <code>R/W</code> . <b>This is where the source is:</b> the assistant reads and builds it here. |

The buggy `HELLO.ASM` lives **on the disk**, not loose in the folder — that is what the assistant reads with `W` and builds with `ASM` . (In the source tree a pristine copy sits beside this file to rebuild the disk from; it is not part of the shipped package.)

**There is no undo.** The assistant writes to the disk when it reassembles, overwriting the buggy `HELLO.ASM` on it with the fix. Copy this folder first if you want the planted bug back without a `git checkout` .